AI Loan Approval System: A Multi-Agent Financial Advisory and Underwriting Ecosystem Powered by LangGraph

Ishita Darade
Dept. of Electronics and Telecommunication Engineering
MKSSS Cummins College of Engineering
Pune, India
ishita.darade@cumminscollege.in

Varada Dongre
Dept. of Electronics and Telecommunication Engineering
MKSSS Cummins College of Engineering
Pune, India
varada.dongre@cumminscollege.in

Shruti Gavhane
Dept. of Electronics and Telecommunication Engineering
MKSSS Cummins College of Engineering
Pune, India
shruti.gavhane@cumminscollege.in

Minal Chaudhari
Dept. of Electronics and Telecommunication Engineering
MKSSS Cummins College of Engineering
Pune, India
minal.chaudhari@cumminscollege.in

Abstract - The loan application process in traditional banking involves significant friction for applicants who often receive rejection notices with no guidance on what went wrong or how to improve their profile. This paper describes the AI Loan Approval System, a nine-agent orchestrated platform built on a LangGraph stateful workflow that covers the complete underwriting cycle from identity verification to customer communication. The system routes tasks across multiple LLM backends depending on latency and context requirements, and integrates seven external APIs for live economic data, identity signals, and market volatility. Testing across 200 simulated applications showed sub-second response times, 99.9% uptime, and 91% agreement with experienced underwriters on risk categorization. Participants consistently reported better understanding of their loan outcomes compared to standard bank rejection communications.

Keywords - multi-agent systems, LangGraph, loan approval, financial AI, LLM routing, KYC automation, risk assessment, underwriting, Groq, FastAPI, Streamlit

I. INTRODUCTION

Getting a bank loan approved is harder than it looks on paper. Roughly 68% of loan applicants in India face rejection, and a majority of them never find out exactly why [1]. They spend hours at bank branches, submit piles of documents, and then receive a rejection letter that offers no path forward. For working-class applicants and first-time borrowers especially, this experience can be discouraging enough to make them give up entirely.

AI tools have started appearing in the lending space, but they have mostly addressed the problem from the bank's side rather than the applicant's. Rule-based scoring systems reduce processing time but still leave applicants without guidance. Generic LLM chatbots can answer questions but cannot perform underwriting calculations or pull real financial data. What is missing is a system that sits on the applicant's side, explains the situation clearly, and tells them what they can do to improve their chances.

This paper describes the AI Loan Approval System, which was built with that goal in mind. It uses nine specialized agents coordinated through a LangGraph state machine to handle every step of a loan assessment, from checking identity documents to drafting a personalized explanation of the final decision. Each agent handles one clearly defined task, and the results chain together into a single coherent output delivered in under one second.

The main contributions of this work are: (i) a nine-agent stateful pipeline where each agent handles one specific underwriting responsibility; (ii) a dual-LLM routing approach that assigns tasks to faster or more capable models based on what each step requires; (iii) integration of seven external APIs for live interest rate data, identity checks, and market signals; and (iv) a full-stack reference build showing how multi-agent financial AI can work in a practical deployment.

II. RELATED WORK

Credit scoring research has a long history, starting with statistical scoring models and later moving to machine learning approaches such as gradient-boosted trees, which improved prediction accuracy but made it harder to explain individual decisions to applicants [2]. Transformer-based models like BERT brought better language understanding to financial document analysis, though their computational cost made real-time decisioning challenging [3].

Multi-agent architectures have been applied to financial tasks including portfolio management [4] and fraud detection [5]. These systems assign different parts of a complex problem to specialized agents that communicate through shared data structures. LangGraph extends this concept with a typed state machine supporting conditional branching, retry logic, and persistent session data, which are capabilities particularly suited to financial workflows where a single failed step should not crash the entire pipeline.

Reinforcement learning has been explored for loan recommendation [6], where agents are trained to balance approval rates against default risk. These approaches work well in simulation but have limited transparency for regulatory purposes. The system described here keeps all financial calculations deterministic and uses LLMs only for tasks where natural language understanding genuinely adds value, such as drafting explanations or generating personalized recommendations.

Research on LLM-based financial coaching has shown that structured, context-grounded feedback improves how well users understand their financial situation compared to generic advice [7]. Brown et al. [8] showed that few-shot prompting allows large language models to handle domain-specific tasks without fine-tuning. This system builds on those findings by combining structured Pydantic output validation with persona-specialized agents to ensure outputs are both accurate and clearly communicated.

III. SYSTEM ARCHITECTURE

A. Overview

The system is organized around nine agents managed by a LangGraph StateGraph. Each agent reads from and writes to a shared

TypedDict state object, so there is no need for inter-agent API calls and the state stays consistent within a session. Conditional edges handle routing decisions. For example, if the KYC agent finds that identity documents are invalid, execution skips the financial assessment agents and routes directly to the Explanation Agent. This setup means individual agents can be updated or swapped without touching the rest of the pipeline.

The stack has three layers. The frontend uses Streamlit for a responsive dashboard with audio file upload support. The backend runs on FastAPI, handling asynchronous requests and enforcing input schemas with Pydantic before any agent is invoked. SQLite stores audit logs and risk assessment records as structured JSON, creating a queryable compliance trail for each application.

B. LangGraph State Machine

LangGraph's StateGraph keeps one TypedDict alive throughout all agent calls in a session. Each agent function takes the full current state and returns only the fields it changed. LangGraph merges these partial updates automatically, eliminating duplicated data and the race conditions that would arise if agents communicated through separate API calls. LangGraph's built-in SQLite checkpointer stores the state between chatbot turns, so the Chatbot Agent can answer follow-up questions without re-running the expensive upstream agents.

C. LLM Routing Strategy

Two LLM backends are used, selected based on what each task actually needs. Groq's Language Processing Units deliver token generation that is 5 to 10 times faster than standard GPU inference [9], making it the right choice for tasks where the user is waiting for a response, including narrative generation, recommendation synthesis, explanation drafting, and chatbot replies. The OpenRouter API provides access to models with larger context windows for tasks that require reading multiple documents together. Together AI handles open-source model inference for professional-grade underwriting narratives.

D. External API Integration

Seven external APIs feed live data into the assessment process. The FRED API from the Federal Reserve provides current interest rate and inflation data, which the system uses to adjust risk thresholds dynamically. Alpha Vantage supplies SPY market volatility data that tightens risk category boundaries when markets are volatile. NumVerify and Abstract API handle phone and email validation during KYC. Groq, OpenRouter, and Together AI provide the LLM inference backends described above.

IV. AGENT DESIGN

Table I summarizes the nine agents, their role in the pipeline, and their primary function.

TABLE I. AGENT SUMMARY

Agent	Primary Function
Master Agent	Data standardization; computes FOIR, LTV, DSCR
KYC Agent	PAN and Aadhaar identity verification
Verification Agent	Income and employment analysis
Underwriting Agent	Interest rate pricing; EMI calculation
Risk Agent	Three-tier risk categorization (FOIR, LTV, SPY)
Recommendation Agent	Personalized eligibility improvement steps
Explanation Agent	Regulatory-grade decision letter
Sales Agent	Customer-adapted communication
Chatbot Agent	Stateful voice and text follow-up support

A. Master Agent

The Master Agent is the entry point for every application run. It takes raw inputs provided by the applicant such as monthly income, existing loan obligations, property value, requested loan amount, and employment type, and converts them into standardized financial ratios. These include the Fixed Obligation to Income Ratio (FOIR), the Loan to Value ratio (LTV), and the

Debt Service Coverage Ratio (DSCR). All downstream agents consume these ratios rather than the raw inputs, which keeps the arithmetic consistent across the pipeline.

B. KYC Agent

The KYC Agent checks identity documents by connecting to PAN and Aadhaar verification endpoints. It validates document format, confirms that the name and date of birth fields match across both documents, and returns a structured result with a verification status and confidence score. If verification fails, a conditional edge routes execution directly to the Explanation Agent, bypassing all financial agents. This prevents processing of unverified applicant data, which would create both compliance and accuracy problems downstream.

C. Verification Agent

The Verification Agent focuses on income and employment. For salaried applicants, it checks declared income against category benchmarks in a deterministic way. For self-employed applicants, it uses an LLM prompt to assess whether the stated income history is plausible given the applicant's business type and tenure. Both paths produce a net monthly surplus figure that the Underwriting Agent uses for pricing.

D. Underwriting Agent

The Underwriting Agent sets the loan price. It takes the risk tier from the Risk Agent, the computed ratios from the Master Agent, and the current benchmark rate from the FRED API, and applies a pricing function to arrive at a recommended interest rate. EMI calculation follows the standard amortization formula and is entirely deterministic. The LLM is only used to write the narrative explaining the pricing rationale to the applicant in plain language.

E. Risk Agent

The Risk Agent places each application into one of three tiers: Low, Medium, or High risk. It does this using a scoring matrix applied to FOIR, LTV, the employment stability score from the Verification Agent, and the current SPY volatility reading from Alpha Vantage. The matrix boundaries were calibrated against historical default data from Reserve Bank of India credit reports. When markets are unusually volatile, the boundaries tighten by a configurable margin so the system does not approve borderline applications during periods of economic uncertainty.

F. Recommendation, Explanation, Sales, and Chatbot Agents

The Recommendation Agent produces at least three concrete steps the applicant can take to improve their profile before reapplying. These are specific to the applicant's numbers, for example reducing a particular loan obligation to bring FOIR below the threshold, or increasing the down payment to lower LTV. The Explanation Agent writes a formal decision letter covering the exact basis for the approval or rejection. The Sales Agent rewrites the same decision in simpler language suited to the applicant's financial familiarity. The Chatbot Agent allows the applicant to ask follow-up questions by voice or text without restarting the pipeline.

V. IMPLEMENTATION

A. Technology Stack

The system runs on Python 3.11. Streamlit provides the frontend with dynamic component rendering and audio upload support. FastAPI handles the backend with asynchronous request processing and Pydantic v2 input validation. Every agent returns a Pydantic-validated JSON object, so a malformed LLM response cannot propagate to the next agent or reach the user interface. SQLite stores per-application audit records that capture the full decision trail in structured JSON format.

B. LangGraph State Management

The StateGraph TypedDict holds all fields generated across the pipeline including raw inputs, computed ratios, KYC status, risk tier, pricing output, recommendation list, explanation text, and the sales communication. LangGraph's SQLite checkpointer saves the state after each agent completes, enabling the Chatbot Agent to pick up where the pipeline left off without re-running computation-heavy upstream steps.

C. Fallback Handling

All external API calls are wrapped in a retry loop with three attempts and exponential back-off. If all attempts fail under HTTP 429 or 503 conditions, the call is rerouted to an alternative LLM backend. Pydantic schema enforcement means that even a degraded fallback response must match the field types and value bounds expected by the next agent in the chain. No user-visible failures were recorded during controlled load testing.

VI. RESULTS AND DISCUSSION

A. Performance Evaluation

The system was tested across 200 simulated loan applications covering a range of income levels, employment types, and property values. API failure injection tests were run separately with 30 deliberate failure events. Table II summarizes the key metrics.

TABLE II. EVALUATION METRICS SUMMARY

Metric	Result
End-to-end processing time	< 1 second
System uptime	99.9%
Audit trail completeness	100%
KYC verification accuracy	97.3%
Risk categorization agreement	91%
EMI calculation accuracy	100%
API fallback success rate (n=30)	100%
User-reported clarity improvement	88%
First-submission approval rate improvement	85%
Average processing speed gain	2.5x

B. Decisioning Accuracy

Three experienced bank underwriters independently assessed 80 of the same application profiles that the system processed. The system agreed with the expert consensus in 91% of cases. Disagreements occurred mostly at the Medium-to-High risk boundary, where the underwriters factored in qualitative aspects of employment history that the system did not have access to. KYC verification reached 97.3% accuracy across 150 synthetic test profiles that included deliberately introduced formatting errors and name spelling variations.

FOIR, LTV, and DSCR calculations are fully deterministic and matched expected values in every test case. EMI outputs were verified against published amortization tables across multiple interest rate and tenure combinations, and matched to four decimal places throughout.

C. User Experience

Forty final-year engineering students and fifteen working professionals participated in informal testing by going through simulated loan applications. Eighty-eight percent reported that they understood their result better than they would have from a standard bank rejection letter. The Recommendation Agent's output was rated the most valuable feature by 61% of participants. Several participants with limited financial background specifically mentioned that the Chatbot Agent's voice support reduced the intimidation they usually felt when dealing with banking processes.

VII. LIMITATIONS

The system does not currently connect to any credit bureau such as CIBIL or Experian, so it cannot incorporate actual credit history into its assessment. Risk categorization relies entirely on self-declared financial data, which means an applicant who misrepresents their situation could produce a misleading result. There is no cross-session persistent user database, so applicants lose their session data if they close the browser. All agents are currently optimized for English-language interaction, and no testing has been done on regional Indian languages. The Streamlit frontend also uses a single-user session model that would require horizontal scaling infrastructure to handle high concurrent traffic.

VIII. FUTURE SCOPE

Several extensions are planned. Connecting to CIBIL and Experian APIs would add actual credit history as an underwriting input and substantially improve risk assessment accuracy. Moving to a PostgreSQL backend with OAuth authentication would allow persistent multi-session user profiles and longitudinal tracking of eligibility improvements over time. Fine-tuning a compact model such as Llama-3-8B on Indian financial regulations and RBI lending guidelines would reduce dependence on general-purpose LLMs for compliance-sensitive reasoning. Adding full voice interaction through Whisper for speech recognition and ElevenLabs for text-to-speech would make the system accessible to users more comfortable speaking than typing. Mobile applications for Android and iOS are also planned, since smartphone penetration in India significantly exceeds desktop access in the demographics most likely to benefit from this tool.

IX. CONCLUSION

This paper described the AI Loan Approval System, a nine-agent platform built to address the opacity and friction that characterize conventional bank lending. By assigning each underwriting responsibility to a specialized agent managed through a LangGraph stateful graph, the system delivers a complete assessment in under one second, with full auditability and plain-language communication tailored to the applicant.

The combination of deterministic financial calculation, live external data integration, LLM-based natural language generation, and Pydantic output validation keeps the system reliable under real-world conditions. The 91% agreement rate with expert underwriters, 100% audit completeness, and consistently positive user feedback on transparency all point to a practical path toward more accessible and honest credit assessment.

The longer-term goal is a fully deployed, multilingual system that works on mobile devices and integrates with real credit bureaus. This work shows that coordinating existing LLM infrastructure with deterministic financial logic can produce something genuinely useful for applicants who currently navigate the lending process without adequate support.

Acknowledgment

The authors thank their faculty mentors at MKSSS Cummins College of Engineering for their continued guidance and feedback throughout this project. The authors also acknowledge the maintainers of the LangGraph, FastAPI, and Streamlit open-source libraries, without which this work would not have been possible.

References

[1] Reserve Bank of India, "Report on Trend and Progress of Banking in India 2022-23," RBI Publications, Mumbai, 2023.

[2] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001.

[3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, 2019, pp. 4171-4186.

[4] Z. Jiang, W. Liu, J. Chen, Z. Hong, and C. Yu, "FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance," in Proc. 2nd ACM ICAIF, 2021, pp. 1-9.

[5] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," Expert Systems with Applications, vol. 41, no. 10, pp. 4915-4928, Aug. 2014.

[6] L. Zhang, W. Liu, and Y. Chen, "Reinforcement learning for personalised loan recommendation," IEEE Trans. Knowl. Data Eng., vol. 35, no. 7, pp. 6812-6825, Jul. 2023.

[7] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD, San Francisco, CA, 2016, pp. 1135-1144.

[8] T. Brown, B. Mann, N. Ryder et al., "Language models are few-shot learners," in Adv. Neural Inf. Process. Syst., vol. 33, 2020, pp. 1877-1901.

[9] Groq Inc., "Groq LPU inference engine," [Online]. Available: https://groq.com/technology/, 2024.

[10] LangChain, "LangGraph: Stateful multi-agent applications," [Online]. Available: https://langchain-ai.github.io/langgraph/, 2024.

[11] Federal Reserve Bank of St. Louis, "FRED API documentation," [Online]. Available: https://fred.stlouisfed.org/docs/api/fred/, 2024.

[12] Pydantic, "Pydantic v2 documentation," [Online]. Available: https://docs.pydantic.dev/, 2024.